



הצעת פרויקט לעבודת גמר

יד הנדסאי תוכנה (שאלון 714918)

בנושא

מתכנן משלוחים ומסלולים

Delivery & Route Planner



שם הסטודנט: **יהב רבינוביץ**

ת"ז: **328456348**

מנחה: **מר אילן פרץ**

תאריך: **1.13.2026**

שם המכללה: **כנפי רוח קריית נוער ירושלים (סמל מוסד: 140129)**

תוכן העניינים

3.....	תיאור הנושא.....
3.....	רקע תיאורטי.....
6.....	תיאור הפרויקט.....
8.....	הגדרת הבעיה האלגוריתמית.....
12.....	הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות.....
14.....	הליכים עיקריים בפתרון למידת מכונה - לא רלוונטי.....
14.....	הליכים עיקריים בתחום מע' הפעלה/רשתות/תקשורת/אבטחה - לא רלוונטי.....
14.....	תיאור פרוטוקולי תקשורת.....
15.....	פיתוחים עתידיים.....
16.....	תיאור טכנולוגיות הנדסה.....
16.....	מסד נתונים.....
19.....	פרטים פורמליים.....

1. תיאור הנושא

עולם הלוגיסטיקה עוסק בניהול, תכנון ובקרה של תהליכי הובלה והפצה של סחורות ממחסנים ומרכזים לוגיסטיים אל לקוחות הקצה. בשנים האחרונות תחום זה הפך למורכב במיוחד עקב גידול משמעותי בכמות ההזמנות, דרישה לאספקה מהירה, תנועה עירונית עמוסה ואילוצים תפעוליים רבים.

הצורך ההנדסי המרכזי בתחום הלוגיסטיקה הוא תכנון מסלולי משלוחים בצורה אופטימלית. יש צורך במערכת חישובית שתדע לחלק הזמנות בין מספר רכבי משלוח, לקבוע סדר ביקורים אצל לקוחות, ולעמוד במגבלות כמו קיבולת רכב, זמני הגעה נדרשים, מרחקי נסיעה וזמני עבודה.

כיום, למרות ההתקדמות הטכנולוגית, ארגונים רבים עדיין נתקלים בקשיים משמעותיים בשל הסתמכות על תכנון ידני או על כלים פשוטים שאינם מסוגלים לעבד את כלל האילוצים בזמנית. חוסר היכולת לבצע אופטימיזציה מלאה של המסלולים מוביל לבזבז דלק, שחיקה מואצת של כלי הרכב, עומס על הנהגים ופגיעה ברמת השירות הניתנת ללקוח. מצב זה מדגיש את הפער הקיים בין הדרישה הגוברת למשלוחים מהירים לבין היכולת המעשית לנהל את המערך הלוגיסטי בצורה יעילה וחסכונית.

2. רקע תיאורטי

בעיית ניתוב כלי הרכב, המוכרת בספרות המקצועית כ-Vehicle Routing Problem או בקיצור VRP, מהווה את אחת מאבני היסוד בתחום חקר הביצועים והאופטימיזציה הקומבינטורית. הבעיה, מגדירה אתגר תכנוני שבו צי רכבים נדרש לספק שירות לקבוצת לקוחות המפוזרים גיאוגרפית, תוך התחלה וסיום בנקודת מוצא אחת או יותר המכונה מחסן לוגיסטי. המטרה המרכזית ב-VRP היא מזעור העלות הכוללת של התפעול, הנמדדת לרוב במונחי מרחק נסיעה כולל, מספר כלי הרכב המופעלים או זמן העבודה הכולל, וכל זאת תחת עמידה במכלול של אילוצים תפעוליים. מבחינה מבנית, ה-VRP נחשבת להרחבה מורכבת של בעיית הסוכן הנוסע, שכן בעוד שהסוכן הנוסע מתמקד במסלול יחיד העובר בכל הנקודות, ה-VRP מחייב חלוקה חכמה של המשימות בין מספר רכבים, מה שיוצר בעיית הקצאה ובעיית סדר בזמנית.

המורכבות החישובית של ה-VRP מסווגת כבעיית NP-Hard, מה שמעיד על כך שלא קיים אלגוריתם המסוגל למצוא פתרון אופטימלי המובטח בזמן ריצה יעיל עבור בעיות בקנה מידה רחב.

ככל שמספר הצמתים והרכבים גדל, מספר השילובים האפשריים של המסלולים צומח בקצב מהיר במיוחד, מה שהופך סריקה מלאה של כלל האפשרויות לבלתי ישימה מבחינה חישובית. בשל כך, פותחו לאורך השנים וריאציות רבות של הבעיה המשקפות אילוצים מהעולם האמיתי, כמו VRP עם מגבלות קיבולת שבו לכל רכב יש מטען מקסימלי, או VRP עם חלונות זמן הדורש הגעה ללקוחות בטווח שעות מוגדר. אילוצים אלו הופכים את מרחב הפתרונות האפשריים למצומצם וקשה יותר לניווט, שכן פתרון שנראה קצר מבחינת מרחק עלול להיפסל עקב איחור ללקוח או חריגה ממשקל המטען המותר.

במישור הפתרונות, הגישות מתחלקות בין שיטות מדויקות לשיטות היוריסטיות. השיטות המדויקות, כמו Branch and Cut או תכנון דינמי, עושות שימוש במודלים מתמטיים קשיחים כדי להגיע לפתרון האופטימלי המוחלט. תכנון דינמי, למשל, מפרק את הבעיה לתתי-בעיות קטנות ושומר את פתרונותיהן כדי להימנע מחישובים חוזרים, אך ב-VRP מורכב כמות תתי-הבעיות הופכת לבלתי נשלטת במהירות. שיטת ה-Branch and Cut משלבת עץ חיפוש עם אילוצים המקצצים חלקים ממרחב הפתרונות שאינם רלוונטיים, אך גם היא מוגבלת לבעיות בעלות מספר לקוחות קטן יחסית. מגבלות אלו הובילו לפיתוחן של מטה-היוריסטיקות, שהן מסגרות עבודה גמישות לחיפוש פתרונות איכותיים בזמן מהיר.

בין האלגוריתם שנבחנו, ניתן למצוא את האלגוריתם הגנטי, השואב השראה מתהליכי האבולוציה בטבע כדי לשפר בהדרגה אוכלוסייה של פתרונות אפשריים. האלגוריתם מבצע פעולות של סלקציה של הפתרונות הטובים ביותר, הצלבה ביניהם ליצירת מסלולים חדשים ומוטציות אקראיות המונעות מהחיפוש להיתקע בפתרונות "אופטימום מקומי". שיטות נוספות כמו חיפוש טאבו משתמשות במנגנוני זיכרון כדי למנוע חזרה על מסלולים שכבר נבדקו, בעוד שחישול מדומה מאפשר קבלת פתרונות פחות טובים בשלבים מוקדמים כדי לחקור אזורים חדשים במרחב הפתרונות. היוריסטיקות פשוטות יותר, כמו אלגוריתם קלארק-רייט, מתמקדות בבנייה מהירה של פתרון ראשוני על בסיס חיסכון במרחק, מה שמהווה לעיתים קרובות נקודת מוצא לשיפור על ידי אלגוריתמים מורכבים יותר. השילוב בין הבנה תיאורטית של מבנה הבעיה לבין בחירה נכונה של אלגוריתם חיפוש הוא המפתח להתמודדות עם האתגרים הלוגיסטיים המודרניים.

להלן טבלה המסבירה על היתרונות והחסרונות של כל אחד מהאלגוריתמים שנבחרו

שם הגישה	תיאור המנגנון	יתרון מרכזי	חסרון משמעותי
Branch and Cut	חלוקת הבעיה לעץ החלטות והוספת חסמים מתמטיים.	הבטחה מתמטית לפתרון הטוב ביותר האפשרי.	דרישות זיכרון וזמן חישוב קיצוניות בבעיות גדולות.
Dynamic Programming	פירוק למשימות קטנות ושמירת תוצאות ביניים בזיכרון.	יעילות גבוהה במציאת פתרונות אופטימליים לתת-מסלולים.	קריסה חישובית ככל שמספר הלקוחות והאילוצים גדל.
Clarke-Wright Savings	איחוד מסלולים בודדים בהתבסס על מקסימום חיסכון במרחק.	מהירות ביצוע כמעט מיידית גם בבעיות רחבות.	פתרון "תאב בצע" (שנוטה Greedy) לפספס אופטימום גלובלי.
Tabu Search	חיפוש מקומי המשתמש ברשימת איסור ("טאבו") על מהלכים קודמים.	מניעה יעילה של חזרה למעגלים סגורים בחיפוש.	תלות גבוהה בכיול פרמטרים וגודל רשימת הזיכרון.
Simulated Annealing	חיקוי תהליך קירור פיזיקלי המאפשר הרעה זמנית בפתרון.	יכולת חקירה מצוינת של מרחבי פתרון מורכבים ולא רציפים.	הגעה לפתרון טוב עלולה לקחת זמן רב עקב תהליך ה"קירור".
אלגוריתם גנטי	שיפור אוכלוסיית מסלולים דרך הצלבה, מוטציה וברירה טבעית.	גמישות רבה בטיפול באילוצים מרובים במקביל.	מורכבות בייצוג של "כרומוזום" המייצג מסלול חוקי.

לאחר בחינת כל האלגוריתמים שנבדקו, האלגוריתם שנבחר עבור המערכת הוא האלגוריתם הגנטי. הבחירה באלגוריתם גנטי נובעת מהאופי של בעיית תכנון המשלוחים, הדומה במהותה לבעיית ניתוב רכבים. (VRP) מדובר בבעיה מורכבת מאוד, שבה מספר האפשרויות גדל בצורה משמעותית ככל שמספר הלקוחות והרכבים עולה. במצבים כאלה, חיפוש ממצה של כל הפתרונות האפשריים אינו ישים בזמן סביר, ולכן נדרש אלגוריתם המסוגל להתמודד עם מרחב פתרונות עצום בצורה חכמה ולא ממצה.

האלגוריתם הגנטי בולט במיוחד למשימה זו ממספר סיבות מרכזיות:

- הוא מאפשר עבודה עם פתרונות מורכבים, הכוללים תיאום של מספר רכבים בזמנית ולא מסלול יחיד בלבד.
- ניתן לשלב בו בקלות אילוצים שונים כגון קיבולת רכב, חלונות זמן, איזון עומסים בין רכבים ומגבלות תנועה.
- האלגוריתם אינו נתקע בקלות בפתרון מקומי, משום שפעולות החיבור והמוטציה מאפשרות חקירה של אזורים חדשים במרחב הפתרונות.
- הוא מתאים לעבודה בסביבה דינמית, שבה הנתונים עשויים להשתנות (כגון הזמנות חדשות או ביטולים), וניתן להריץ אותו מחדש או להמשיך תהליך שיפור קיים.

בשל תכונות אלו, אלגוריתם גנטי מספק איזון טוב בין איכות הפתרון לבין זמן החישוב, ומהווה בחירה מתאימה במיוחד לפתרון בעיות לוגיסטיות מורכבות כמו תכנון וניהול מסלולי משלוחים. האלגוריתם אינו מבטיח פתרון אופטימלי מוחלט, אך בפועל מצליח להגיע לפתרונות איכותיים מאוד, העומדים בדרישות ההנדסיות של המערכת ומאפשרים יישום מעשי בעולם האמיתי.

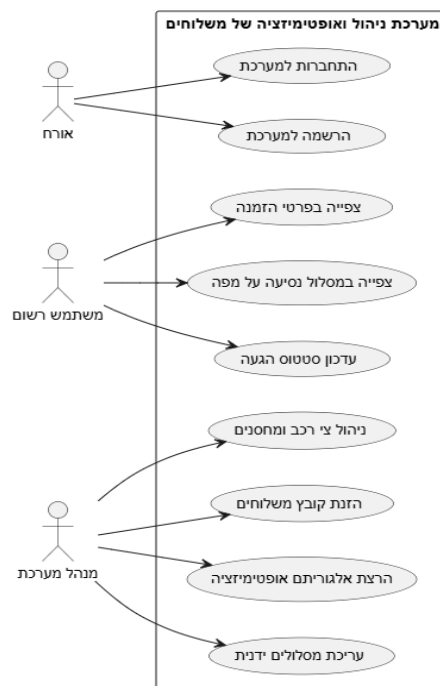
3. תיאור הפרויקט

המערכת המפותחת במסגרת פרויקט זה היא פלטפורמה חכמה לניהול ואופטימיזציה של מערכי משלוחים, המיועדת לעסקים המפעילים צי רכב להפצת סחורות בסביבה עירונית.

המערכת משמשת ככלי עזר למנהלי לוגיסטיקה וסדרני הפצה, ומאפשרת להם להפוך את תהליך תכנון המסלולים הידני והמורכב לתהליך אוטומטי, מהיר ומדויק. השימוש במערכת מתבצע באמצעות ממשק חכם אליו מוזנים נתוני ההזמנות, כתובות הלקוחות ומשאבי הצי הזמינים, ובתמורה מפיקה המערכת תוכנית עבודה אופטימלית המחלקת את המשימות בין הנהגים השונים בצורה מאוזנת.

הרכיבים העיקריים של המערכת מנקודת מבט המשתמש כוללים מודול קליטת נתונים, מנוע חישוב מבוסס אלגוריתם גנטי וממשק תצוגה גיאוגרפי. המשתמש מקיים אינטראקציה עם המערכת על ידי הגדרת פרמטרים תפעוליים, כגון קיבולת המכוניות או חלונות זמן נדרשים, וצופה בתוצאות על גבי מפה אינטראקטיבית המציגה את מסלולי הנסיעה המפורטים לכל רכב. המערכת פותרת את האתגרים שהוצגו בסעיף הראשון על ידי החלפת התכנון הידני בחישוב המבוסס על מודלים מתמטיים מתקדמים; היא מצמצמת משמעותית את הנסועה המיותרת, מבטיחה עמידה באילוצי זמן קשיחים ומונעת מצבים של עומס יתר על נהג בודד בעוד אחרים נותרים ללא פעילות.

תרשים Use Case של המערכת:



הסבר קצר על כל מקרה של המערכת:

- **התחברות למערכת (Guest Login / System Login)**
מאפשר למשתמש (אורח או רשום) להתחבר למערכת כדי להשתמש בשירותים שלה בהתאם להרשאותיו.
- **הרשמה למערכת (Register)**
מאפשר לאורח ליצור חשבון חדש במערכת ולהפוך למשתמש רשום.
- **צפייה בפרטי הזמנה (View Order Details)**
מאפשר למשתמש רשום לראות את פרטי ההזמנה שלו, כגון מוצרים, כמות, כתובת משלוח ומועד צפוי.

- **צפייה במסלול נסיעה על מפה (View Route on Map)**
מאפשר למשתמש רשום לראות את מסלול המשלוח שלו בצורה גרפית על מפה, כולל נקודות עצירה וזמנים משוערים.
- **עדכון סטטוס הגעה (Update Arrival Status)**
מאפשר למשתמש רשום או למנהל לעדכן את סטטוס ההגעה של משלוח, לדוגמה: "בתהליך", "נמסר", "מושהה".
- **ניהול צי רכב ומחסנים (Manage Fleet and Warehouses)**
מאפשר למנהל המערכת לנהל את צי הרכב הזמין והמחסנים, כולל הוספה, הסרה או עדכון של רכבים ומחסנים.
- **הזנת קובץ משלוחים (Upload Shipment File)**
מאפשר למנהל להזין מידע על קבוצת משלוחים באמצעות קובץ חיצוני, מה שמיעיל את הטיפול בכמויות גדולות של הזמנות.
- **הרצת אלגוריתם אופטימיזציה (Run Optimization Algorithm)**
מאפשר למנהל להפעיל אלגוריתם שמחשב את חלוקת המשלוחים והמסלולים האופטימליים לצי הרכב, בהתאם לפרמטרים שונים כמו זמן, מרחק וכוח אדם.
- **עריכת מסלולים ידנית (Edit Routes Manually)**
מאפשר למנהל לערוך ולשנות מסלולי משלוחים באופן ידני, במקרה שיש צורך בהתאמות מיוחדות.

4. הגדרת הבעיה האלגוריתמית

הבעיה האלגוריתמית העומדת בליבת הפרויקט מוגדרת כבעיית אופטימיזציה מסוג ניתוב כלי רכב עם מגבלות קיבולת וחלונות זמן, או בשמה המקצועי - Capacitated Vehicle Routing Problem with Time Windows (CVRPTW). הגדרה זו מהווה תרגום מתמטי לאתגרים הלוגיסטיים שתוארו בסעיף הראשון, ובפרט לצורך בניהול יעיל של ציי רכב בסביבה עירונית צפופה. הבעיה דורשת תכנון של קבוצת מסלולים אופטימלית היוצאת מנקודת מוצא אחת, כך שכל לקוח יקבל שירות בדיוק פעם אחת על ידי רכב בודד, וכל זאת תוך מזעור פונקציית המטרה הכוללת את המרחק המצטבר, זמן הנסיעה וצריכת המשאבים.

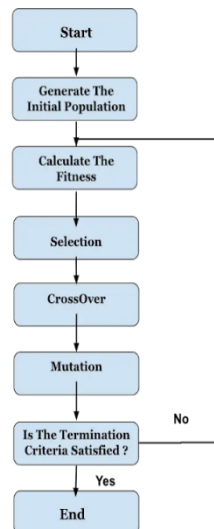
כדי להבין את עומק הבעיה, יש לבחון את המגבלות המרכזיות המשפיעות על האלגוריתם, כאשר הראשונה שבהן היא מגבלת הקיבולת. כל משאית בצי הרכב מחזיקה בנפח או במשקל מקסימלי שהיא יכולה לשאת, מה שמאלץ את המערכת "לחתוך" מסלולים ולחזור למחסן להעמסה מחדש, גם אם גיאוגרפית היה משתלם להמשיך ללקוח הבא. לצד זאת, מגבלת חלונות הזמן מוסיפה רובד של דחיפות, שכן הגעה ללקוח מחוץ לטווח השעות שהוגדר נחשבת לכישלון של המסלול. הקשר הישיר בין הגדרה אלגוריתמית זו לאתגרים שהוצגו בסעיף 1 טמון במורכבות הריאליסטית של עולם הלוגיסטיקה, שבו המערכת נדרשת לתמרן בין צרכים סותרים, כמו רצון למזער נסועה מול הכרח להגיע ללקוחות מרוחקים בשעות ספציפיות.

מבחינה חישובית, הבעיה מוגדרת כבעיית NP קשה - הגדרה שמשמעותה היא שאין כיום אלגוריתם שמסוגל למצוא את הפתרון המושלם ביותר בזמן סביר ככל שמספר הלקוחות גדל. מרחב הפתרונות גדל באופן מעריכי עם הוספת כל לקוח או רכב חדש לצי, מה שהופך סריקה מלאה של כל האפשרויות לבלתי אפשרית מבחינה טכנולוגית. אפיון זה מחייב את המערכת להשתמש בגישות מבוססות חיפוש חכם, כגון האלגוריתם הגנטי. אלגוריתם זה מחקה תהליכים מהטבע כמו ברירה טבעית ומוטציות כדי לגדל פתרונות טובים יותר מדור לדור. במקום לחפש את הפתרון המושלם, המערכת מספקת פתרונות איכותיים וקרובים מאוד לזמן האופטימלי בזמן אמת, מה שמאפשר למנהל המערכת להריץ את האופטימיזציה ולקבל מסלולים מוכנים לעבודה באופן מיידי.

תהליך פיתוח הפתרון כלל מחקר ויישום של אלגוריתמים אבולוציוניים לשיפור הדרגתי של פתרונות, לצד הטמעת מנגנונים לבדיקת אילוצים לוגיסטיים וניהול מסלולים אופטימלי. האלגוריתם שהמערכת תשתמש בו תתייחס לכל פתרון כאל רשימת מסלולים מלאה לכל הרכבים במערכת. כלומר, מערך שמכיל את סדר הלקוחות שכל רכב צריך לבקר בהם. זהו האובייקט שהאלגוריתם מקבל כקלט בכל שלב, ועליו הוא מבצע פעולות של שיפור, סינון, ויצירה מחדש. הפלט מכל שלב הוא שוב פתרון משופר, או קבוצת פתרונות שממשיכה לשלב הבא.

להלן איור המייצג את שלבי האלגוריתם:

Genetic Algorithm Pseudo Code



שלבים המתארים את תהליך העבודה של האלגוריתם המרכזי:

1. יצירת פתרונות התחלתיים: בתחילת התהליך האלגוריתם מקבל כקלט את רשימת כל

הלקוחות והמשלוחים, וממנו הוא מייצר קבוצה ראשונית של פתרונות. כל פתרון הוא הצעה מלאה כיצד לחלק את כל הלקוחות בין הרכבים ובאיזה סדר כל רכב ייסע. בניגוד ליצירה אקראית לחלוטין, האלגוריתם בוחר להכניס כל נקודת עצירה למיקום הגיוני מבחינת מרחק ולוגיסטיקה, כך שכבר בשלב הראשון מתקבלים פתרונות ראשוניים איכותיים יחסית. הפלט של השלב הזה הוא אוסף פתרונות ראשוניים שמוכן להתחלת תהליך השיפור.

2. סינון הפתרונות החלשים: בשלב זה הקלט הוא כל הפתרונות שנוצרו עד כה. עבור כל

פתרון האלגוריתם מחשב מדד איכות. למשל, סך כל אורך המסלולים, זמני נהיגה, ואיזון בין הרכבים. לאחר חישוב הציון, הפתרונות מדורגים מהפתרון הטוב ביותר לגרוע ביותר. האלגוריתם מסנן החוצה את אלו בעלי האיכות הנמוכה ביותר ומשאיר רק את החזקים. הפלט של השלב הוא קבוצה מצומצמת ואיכותית יותר של פתרונות, שממנה ניתן להמשיך ולבנות שיפורים.

3. יצירת פתרונות חדשים מהטובים: כעת הקלט הוא הפתרונות האיכותיים שנותרו לאחר

הסינון. האלגוריתם בוחר פתרון מוביל אחד ופתרון נוסף שנבחר באקראי, ומבצע ביניהם

חיבור (Crossover), כלומר, מחליף ביניהם חלקים שונים של המסלולים כדי לייצר שני פתרונות חדשים. כך מתקבלים פתרונות המכילים שילוב של רעיונות מוצלחים משני פתרונות קיימים. הפלט של השלב הזה הוא סט חדש של פתרונות, המוסיפים גיוון לקבוצה הראשית.

4. בדיקת אפשרות לשינוי אקראי ("מוטציה"): לאחר יצירת פתרונות חדשים, האלגוריתם

בודק על כל פתרון בנפרד האם לבצע שינוי קטן ואקראי. למשל, החלפת סדר שני לקוחות, העברת נקודת עצירה לרכב אחר, או שינוי מיקום של עצירה בשרשרת. הקלט כאן הוא כל פתרון שנוצר זה עתה, והפלט הוא או פתרון שעבר שינוי קטן, או פתרון שנשאר כפי שהיה. מטרת השלב היא לפתוח אפשרויות חדשות ולמנוע מהאלגוריתם להיתקע על רעיונות מוגבלים.

5. יצירת קבוצה קטנה לשיפור מקומי: כאשר מתבצעת מוטציה על פתרון מסוים,

האלגוריתם יוצר סביבו קבוצה קטנה של פתרונות המבוססים עליו. כל הפתרונות בקבוצה הזו עוברים תהליך מואץ של שיפור: סינון, יצירה מחדש, ולעיתים גם שינויים אקראיים נוספים. הקלט כאן הוא פתרון אחד שעבר מוטציה, והפלט הוא פתרון משופר מאוד שמייצג את התוצאה הטובה ביותר של סדרת השיפורים המקומיים. זה מאפשר לאלגוריתם ללטש רעיון מסוים לעומק.

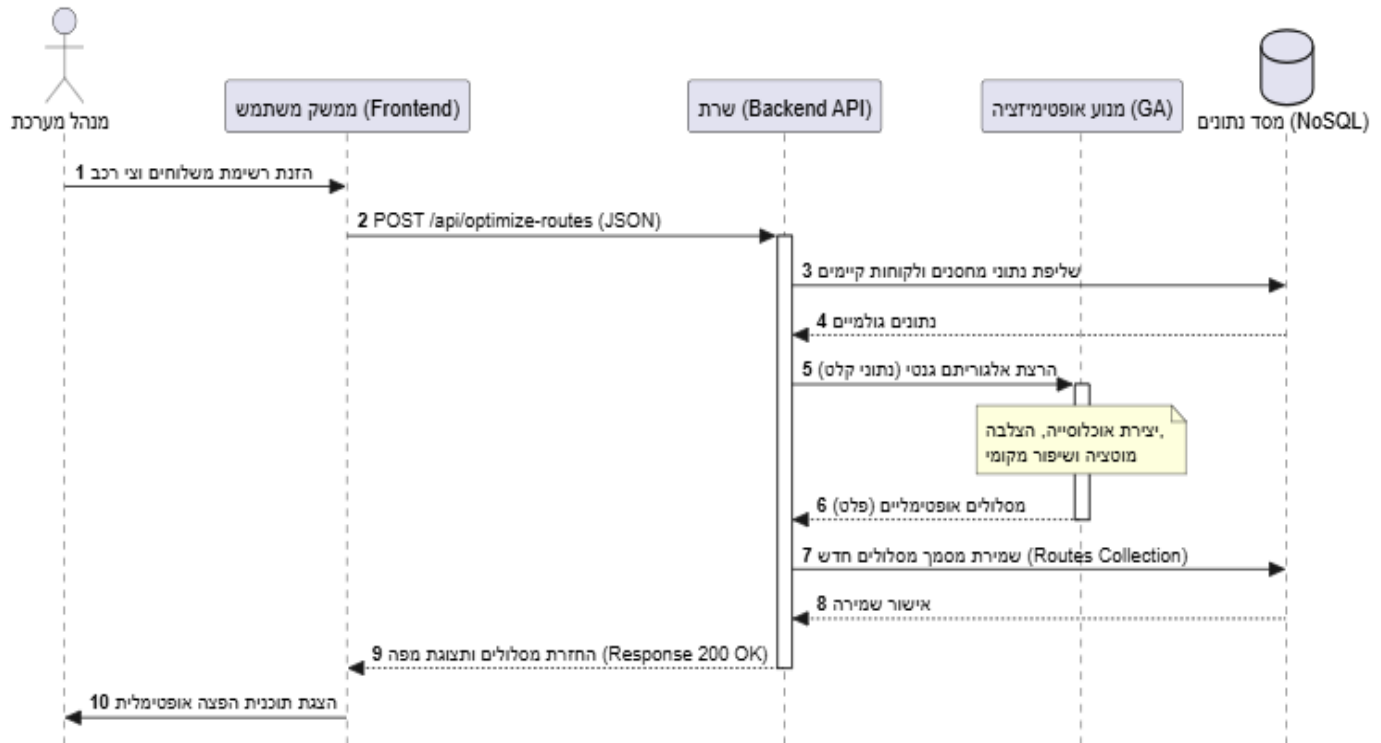
6. הוספת הפתרון המשופר לקבוצה הראשית: לאחר השלמת תהליך השיפור המקומי,

הפתרון הטוב ביותר נוסף חזרה לקבוצה הראשית של הפתרונות. הקלט הוא הפתרון שעבר שיפור מקומי, והפלט הוא קבוצה מעודכנת ועשירה יותר של פתרונות הכוללת גם את החידושים שהתגלו בתהליך המיקוד.

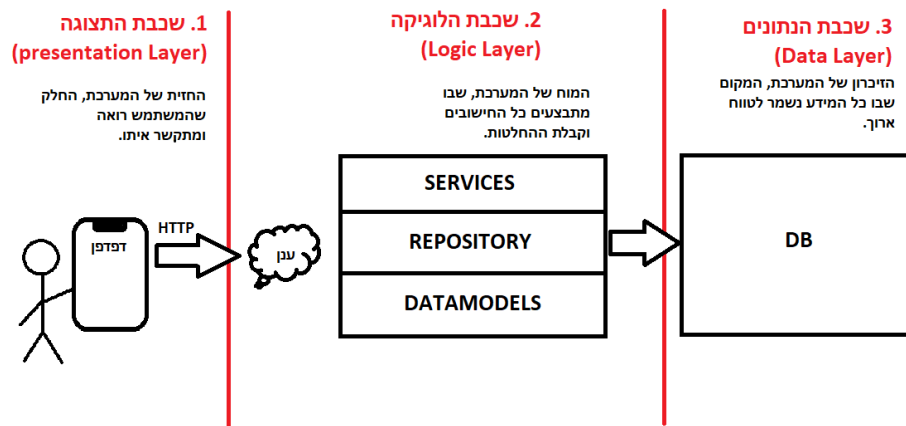
לאחר מכן האלגוריתם חוזר על כל השלבים שוב ושוב: מסנן, יוצר חדשים, משפר, מוסיף, וחוזר. התהליך נעצר רק כאשר כבר הרבה זמן לא הצליח להשתפר, או כשנגמר הזמן שהוקצב לו.

5. הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות

להלן תרשים Sequence Diagram של האלגוריתם המרכזי:



המערכת בנויה לפי **ארכיטקטורת שלוש השכבות**. ארכיטקטורה זו משתלבת בצורה טבעית וישירה בפרויקט ה-Route Planner & Delivery, משום שהיא מאפשרת לחלק את המערכת המורכבת למבנים ברורים שכל אחד מהם אחראי על חלק אחר בתהליך תכנון המסלולים והמשלוחים.



שכבת תצוגה (frontend): היא זו שהמשתמשים עובדים מולה, והיא אחראית להצגת המידע והזנת נתונים כמו לקוחות, הזמנות ובקשה לחישוב מסלול. הממשק לא מבצע חישובים בעצמו. במקום זאת הוא שולח את כל הנתונים לשרת בפורמט JSON דרך API ייעודי. בקשות אלו מתבצעות באמצעות HTTP ומעבירות לשרת מידע מלא, ברור ומובנה. בפרויקט זה, שכבה זו אחראית על הזנת נתונים כגון לקוחות, הזמנות, רכבים ופרטי משלוח, שליחת בקשה לחישוב מסלולי משלוחים, והצגת תוצאות החישוב, כולל מסלולים, סדר ביקורים וזמני הגעה משוערים.

שכבת היישום: שכבה זו היא לב המערכת, ובה נמצאת כל הלוגיקה העסקית. כאן מתבצעים החישובים האופטימליים של מסלולי המשלוחים באמצעות אלגוריתמים לפתרון הבעיה. לאחר החישוב, השרת מחזיר לממשק תשובה, גם כן בפורמט JSON, הכוללת את המסלול האופטימלי, זמני נסיעה משוערים ונתונים נוספים. שכבה זו אחראית על קבלת הנתונים משכבת התצוגה, בדיקת תקינות נתונים ואילוצים (קיבולת, זמני אספקה, עומסים), הרצת האלגוריתם הגנטי לפתרון בעיית ניתוב הרכבים (VRP), חישוב מסלולי משלוחים אופטימליים, והחזרת תוצאות החישוב לשכבת התצוגה.

שכבת הנתונים (database) השכבה שומרת בצורה מסודרת את כל המידע הדרוש למערכת, כמו פרטי הלקוחות, הרכבים, המיקומים וההזמנות. השרת הוא היחיד שמתקשר עם בסיס הנתונים, קורא ממנו נתונים ומעדכן אותו בחישובים ובתוצאות חדשות. שילוב של שלוש השכבות יוצר מערכת נוחה לפיתוח, מאובטחת וניתנת להרחבה, שבה כל שכבה אחראית על חלק ברור בתהליך. בפרויקט זה נשמרים במסד הנתונים נתונים כגון פרטי לקוחות ומיקומם, רכבים וקיבולת, הזמנות ומשלוחים, ומסלולים מחושבים ותוצאות קודמות בנוסף לכך נשתמש גם במודל **שרת לקוח** (Client-Server model) מהווה את התשתית התקשורתית המאפשרת את פעולתה של מערכת תכנון המסלולים, והוא מגדיר את חלוקת התפקידים בין ממשק המשתמש לבין מנוע החישוב.

במודל זה, המערכת מופרדת לשתי ישויות מרכזיות המקיימות ביניהן אינטראקציה מתמדת של בקשה ותגובה. תפקידו של הלקוח הוא לרכז את המידע הגולמי ולשלוח אותו כבקשה מובנית אל השרת, מבלי לבצע את עיבוד הנתונים הכבד בעצמו.

השרת פועל כיחידת העיבוד המרכזית והעוצמתית של המערכת. עם קבלת הבקשה מהלקוח, השרת מפעיל את האלגוריתמים האבולוציוניים המורכבים ומשתמש במשאבי מחשב נרחבים כדי לסרוק מיליוני אפשרויות לסידור מסלולים. ההפרדה הזו חיונית מבחינה הנדסית, שכן היא

מאפשרת למשתמש קצה להפעיל כלים טכנולוגיים מתקדמים גם ממכשירים בעלי כוח עיבוד מוגבל, תוך ריכוז הלוגיקה העסקית והחישובים האינטנסיביים בשרת ייעודי ומאובטח.

התקשורת בין הישויות מתבצעת בפרוטוקול העברת נתונים סטנדרטי, כאשר בסיום תהליך האופטימיזציה, השרת מנסח תגובה המכילה את הפתרון הנבחר ומעביר אותה חזרה ללקוח לצורך הצגה ויזואלית. מבנה זה מבטיח סינכרון מלא בין בסיס הנתונים לבין המשתמשים השונים, ומאפשר למערכת לגדול ולהתרחב תוך שמירה על ביצועים גבוהים ויציבות לאורך זמן

6. הליכים עיקריים בפתרון למידת מכונה - לא רלוונטי

הצעת פרויקט זו אינה עוסקת בפתרון בעיית למידת מכונה, ולכן הוא לא רלוונטי.

7. הליכים עיקריים מע' הפעלה/רשתות/תקשורת/אבטחה-לא רלוונטי

הצעת פרויקט זו אינה עוסקת בתחומי מע' הפעלה/רשתות/תקשורת/אבטחה, לכן לא רלוונטי

הערה חשובה: ההתייחסות למודל שרת לקוח מופיעה בהרחבה בסעיף 5 של ההצעה.

8. תיאור פרוטוקולי תקשורת

- **HTTP:** כדי שהחלקים השונים במערכת יוכלו לדבר אחד עם השני בצורה חלקה, אנחנו משתמשים בפרוטוקול האינטרנט HTTP. אפשר לדמיין את זה כמו שיחה בין לקוח למלצר במסעדה, כאשר הלקוח - האפליקציה שולח בקשה עם הוראות ברורות, והשרת המהווה את המוח של המערכת מעבד אותן ומחזיר תגובה עם הנתונים המבוקשים או אישור שהפעולה הצליחה. התקשורת הזו נעשית בצורה מאובטחת ומסודרת, כך שכל מידע שנשלח מגיע בדיוק לאן שהוא צריך.
- **TCP/IP:** בבסיס של כל התהליך הזה נמצאים פרוטוקולי ה-TCP/IP שתפקידם לדאוג לצד הטכני והלוגיסטי של העברת המידע ברשת. ה-TCP מתפקד כמו שירות משלוחים אחראי המוודא שכל חבילת מידע הגיעה בשלמותה, בסדר הנכון וללא שגיאות, בעוד ה-IP הוא האחראי על הכתובות ומנווט את המידע בין המחשב של המשתמש לבין השרת שלנו ברחבי הרשת. השילוב של כל הכלים האלו מבטיח שהמערכת תהיה יציבה, מהירה ואמינה.

- **Rest API:** מעל שפת התכנות הבסיסית, המערכת משתמשת בשיטה שנקראת Rest API, שמאפשרת לארגן ולשלוח מידע בצורה מסודרת דרך האינטרנט. הרעיון הוא שכל מידע או פעולה במערכת מקבלים כתובת קבועה, שנקראת Endpoint (נקודת קצה), וכשמשתמש או תוכנה שולחים בקשה לכתובת הזו, הם מקבלים את המידע או מבצעים את הפעולה הרצויה. בפרויקט שלנו יהיו מספר מקרים שבהם נשתמש ב Rest API בכדי שהמערכת תוכל לתקשר בין השרת למשתמשים או בין רכיבים שונים בצורה פשוטה, ברורה ומסודרת.
- **Google maps API:** המערכת תתחבר למאגר המידע של גוגל כדי לקבל נתוני מרחק וזמן נסיעה מדויקים, המייצגים את הבסיס לחישוב המסלולים במערכת. נוכל גם להראות ויזואלית בעזרתה את מסלולי הרכבים.

המידע בנקודות הקצה האלו עובר בפורמט שנקרא JSON, שהוא פורמט טקסט פשוט מאוד שנוח גם למחשבים וגם לבני אדם לקרוא. בתוך ה JSON, אנחנו אורזים את כל פרטי ההזמנות, הכתובות של הלקוחות והמסלולים שהאלגוריתם בנה בצורה היררכית ומסודרת.

9. פיתוחים עתידיים

קיימים מספר כיווני התפתחות מרכזיים שיכולים להרחיב את יכולותיה של המערכת במישור העסקי והטכנולוגי כאחד.

- **חיזוי עומסים בלמידת מכונה:** שימוש בבינה מלאכותית כדי ללמוד מהיסטוריית הנסיעות היכן יש פקקים או עיכובי פריקה קבועים, ושילוב המידע הזה בתכנון למניעת איחורים.
- **מסלולים ירוקים וצמצום זיהום:** הוספת שיקול סביבתי שבוחר מסלולים שחוסכים בדלק ופליטת מזהמים, תוך ניתוב רכבים חשמליים למרכזי ערים צפופים.
- **אופטימיזצית העמסה:** הנחיית הנהג כיצד לסדר את החבילות בתוך המשאית לפי סדר הפריקה המתוכנן, כדי לחסוך זמן יקר של חיפוש סחורה בכל עצירה.

10. תיאור טכנולוגיות הנדסה

■ שפות תכנות

- **JAVA 21**: שפת תכנות עילית ומונחית עצמים (OOP) המאפשרת פיתוח מתקדם ומקיף של מערכות תוכנה מסוג Full Stack M, הכוללות גם את צד ה-frontend: ממשקי המשתמשים והאינטרקציות וגם את צד ה-backend: הלוגיקה העסקית, ניהול הנתונים והתקשורת עם מסדי הנתונים. השפה מספקת כלים לניהול עצמים, היררכיות, פולימורפיזם ומודולריות, מה שמקל על בנייה, תחזוקה והרחבה של מערכות מורכבות.

■ מסגרת עבודה תשתיות תוכנה (framework)

- **Spring Boot 3.5.8**: מספקת למפתחי Java תשתית נוחה ומובנית לפיתוח כל רכיבי ה-Backend של מערכת התוכנה. באמצעותה ניתן להקים ולהריץ שרת HTTP (כגון Apache Tomcat), לפתח שכבת Services לניהול והפעלת הלוגיקה העסקית בצד השרת, לנהל את הגישה למסד הנתונים דרך Repositories ולבצע פעולות CRUD, וכן לממש REST Controllers האחראים על קבלת בקשות HTTP, עיבודן ושליחת תגובות מתאימות ללקוח.
- **ממשק משתמש Vaadin 24.9.5**: מאפשרת למפתחי Java ליצור ממשקי משתמש מתקדמים ואינטראקטיביים בצד הלקוח, הפועלים בדפדפן, תוך שימוש בקוד Java בלבד וללא צורך בהתעסקות ישירה עם CSS, HTML או JavaScript.
- Vaadin מספקת אוסף רחב של רכיבי UI מוכנים לשימוש, כאשר עיבוד הלוגיקה מתבצע בשרת, והעדכונים מועברים לדפדפן בזמן אמת באמצעות WebSockets ו-Push.

■ סביבות פיתוח משולבות (IDEs)

- **VSCode 1.108**: סביבת פיתוח משולבת (IDE) המספקת את כל הכלים הדרושים לבניית קוד הפרויקט, ובהם עורך קוד חכם, קומפיילר, כלי לניפוי שגיאות, השלמות קוד אוטומטיות, גישה נוחה לתיעוד ה-API. הסביבה תומכת בניהול פרויקטים באמצעות Maven ומאפשרת הוספת הרחבות שונות, כמו תוסף MongoDB לעבודה מהירה עם מסדי נתונים בענן ותוסף PlantUML ליצירה והמחשה של תרשימי Use Case, Sequence ו-Class.

■ מנגנונים נוספים

- **MongoDB Atlas:** שירות ענן המאפשר ניהול ושליטה מלאה על מסד הנתונים באמצעות הדפדפן. השירות מספק כלים לצפייה ועריכה של הנתונים, ניהול אבטחה על-ידי יצירת משתמשים ומנהלים המוגנים בסיסמה, וכן הגבלת גישה ברמת הרשת באמצעות רשימות כתובות IP מורשות, כך שרק גורמים מאושרים יכולים להתחבר למסד הנתונים.

11. מסד נתונים

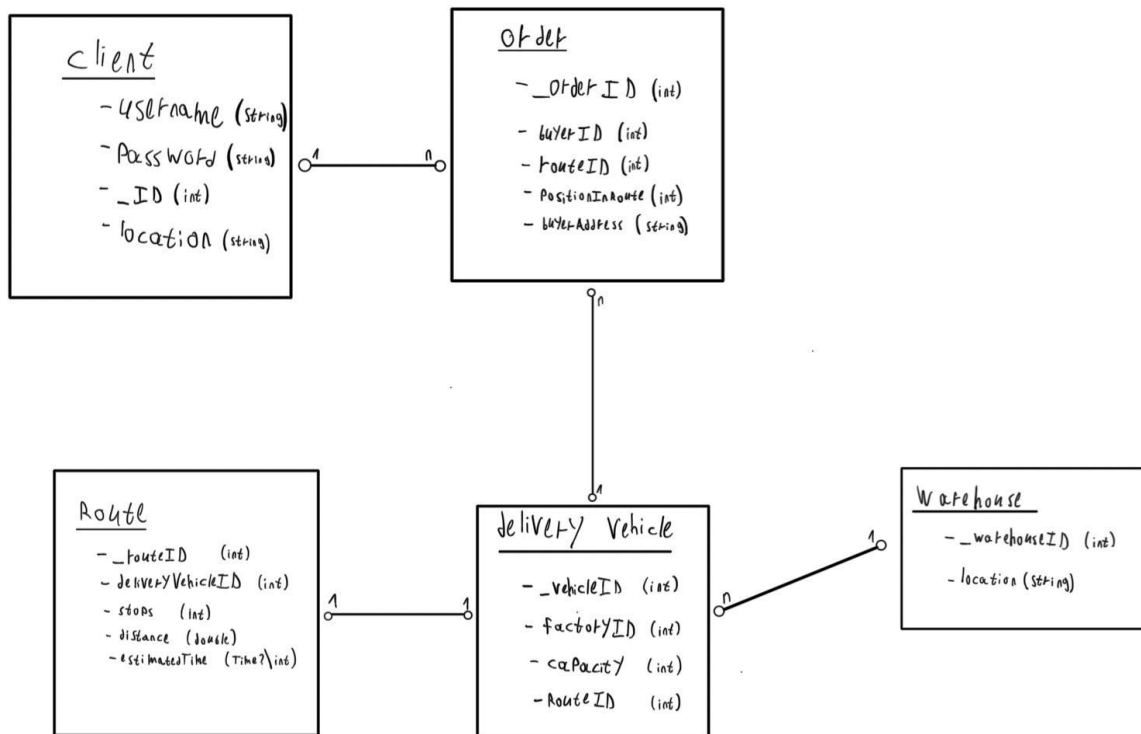
ניהול הנתונים במערכת ה-Route Planner & Delivery מתבצע באמצעות מסד נתונים המאוחסן בתשתית ענן. הבחירה בפתרון ענן נובעת מהצורך בהבטחת זמינות גבוהה של הנתונים, יכולת גדילה מהירה וביצוע גיבויים אוטומטיים השומרים על שרידות המידע ללא צורך בתחזוקת חומרה מקומית. עבור המערכת נבחר שימוש במסד נתונים מסוג NoSQL (כמו MongoDB). בחירה זו נובעת מהאופי הדינמי של הנתונים הלוגיסטיים; מסד נתונים מסוג זה מאפשר גמישות מרבית במבנה המידע, כך שניתן לשמור אובייקטים מורכבים כמו "מסלול נסיעה" המכיל בתוכו רשימה משתנה של הזמנות ומיקומים במסמך אחד, ללא צורך בביצוע פעולות הצלבה (Joins) מורכבות בין טבלאות רבות.

מבנה הנתונים מאורגן באוספים (Collections) המייצגים את הישויות המרכזיות במערכת. הגישה הלא-רציונלית מאפשרת למערכת להתרחב בקלות: אם בעתיד נרצה להוסיף לכל הזמנה נתונים חדשים כמו צילום של החבילה או חתימה דיגיטלית, נוכל לעשות זאת מבלי לשנות את כל מבנה מסד הנתונים. המידע נשמר בפורמט הדומה ל-JSON, מה שיוצר סנכרון מושלם בין הדרך שבה המידע נשמר בבסיס הנתונים לבין הדרך שבה הוא עובר בפרוטוקול התקשורת ועובר עיבוד באלגוריתם.

להלן פירוט האוספים שישמרו במסד הנתונים:

שם האוסף	תיאור השדות המרכזיים	תפקיד במערכת
לקוח (User)	ID, שם משתמש, סיסמה, מיקום גיאוגרפי.	ניהול פרטי משתמשים ונקודות יעד.
מחסן (Warehouse)	ID, מיקום מדויק (קואורדינטות).	הגדרת נקודות היציאה והחזרה של הצי.
רכב (Vehicle)	ID, קיבולת מטען, שיוך למחסן, ID של מסלול נוכחי.	הגדרת המשאבים הזמינים לביצוע המשימות.
הזמנה (Order)	ID, כתובת משלוח, ID משתמש, סטטוס, מיקום בתוך המסלול.	תיעוד דרישות הלקוח ומיקומן ברצף החלוקה.
מסלול (Route)	ID, ID רכב, רשימת עצירות (Array), מרחק, זמן הגעה משוער.	שמירת הפלט הסופי של האלגוריתם הגנטי.

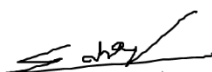
להלן תרשים ERD של הנתונים:



12. פרטים פורמליים**לוחות זמנים:**

לסיים עד תאריך	שלבי עבודה
1.12.2025	בחירת פרויקט, חקירה ולמידה לעומק של נושאי הפרויקט
11.12.2025	כתיבה והגשת הצעת הפרויקט לאישור משרד החינוך
13.1.2026	מימוש הקוד של האלגוריתם המרכזי, ביצוע בדיקות ושיפורים
3.2.2026	בניית צד שרת
17.2.2026	בניית מסד הנתונים ושילובו
3.3.2026	בניית צד לקוח
24.3.2026	כתיבת ספר הפרויקט
7.4.2026	הגשת הפרויקט כולו (ספר + קוד) להגנה וקבלת ציון מגן

מנחה הפרויקט: _____



חתימת הסטודנט: _____

חתימת רכז המגמה: _____